

AUTOMATION TEST REPORT – BLAZEDEMO FLIGHT BOOKING PROJECT

Prepared By

Ashish Shukla

Framework

Selenium WebDriver + Java + TestNG + Maven + Page Object Model (POM)

Project Type

Web Automation Testing

GitHub Repository-

<https://github.com/shu786ash/BlazeDemoAutomation>

1. Project Overview

The BlazeDemo Flight Booking Automation Project is developed to automate the complete flight booking process of the BlazeDemo web application. The primary objective of this project is to reduce manual testing effort, improve test execution speed, increase accuracy, and validate the end-to-end booking workflow through automation.

The framework is built using Selenium WebDriver with Java and follows the Page Object Model (POM) design pattern. The project automates important functionalities such as flight search, flight selection, passenger information entry, booking confirmation, screenshot capture, report generation, and result validation.

The framework also supports Data-Driven Testing using Excel files and generates detailed execution reports using TestNG and Extent Reports. Additionally, Continuous Integration and Continuous Deployment (CI/CD) practices have been implemented using Jenkins and Docker.

2. Objectives of the Project

The main objectives of this automation project are:

- Automate the complete flight booking workflow.
- Reduce repetitive manual testing activities.

- Improve test execution efficiency.
 - Validate application functionality across multiple test data sets.
 - Generate detailed execution reports.
 - Integrate automation execution with Jenkins.
 - Execute automation inside Docker containers.
 - Demonstrate industry-standard automation framework practices.
-

3. Technologies Used

The following technologies and tools were used during the development of this automation framework:

- Java
 - Selenium WebDriver
 - TestNG
 - Maven
 - Apache POI
 - Page Object Model (POM)
 - Git
 - GitHub
 - Jenkins
 - Docker
 - Extent Reports
 - Eclipse IDE
-

4. Framework Architecture

The automation framework follows the Page Object Model (POM) design pattern, which improves code reusability, maintainability, and scalability.

Project Structure

Base Package

Contains browser initialization and teardown methods.

Pages Package

Contains page classes representing application web pages.

- HomePage
- FlightSearchPage

- PurchasePage
- RegistrationPage

Testcases Package

Contains all test execution classes.

- BlazeDemoTest

Utilities Package

Contains reusable utility classes.

- ConfigReader
- ExcelUtils
- ScreenshotUtils
- DataProviders
- ExtentManager

Listeners Package

Contains TestNG listener implementation.

- MyListener

Resources

Contains test data and configuration files.

- TestData.xlsx
 - config.properties
-

5. Test Scenarios Automated

Scenario 1: Search Flights

Description

Verify that users can search flights between selected cities.

Steps

1. Open BlazeDemo application.
2. Select departure city.
3. Select destination city.
4. Click Find Flights.

Expected Result

Available flights should be displayed successfully.

Scenario 2: Select Flight

Description

Verify that users can select a flight from the available flight list.

Expected Result

Selected flight should navigate to the purchase page.

Scenario 3: Complete Flight Booking

Description

Verify that users can complete flight booking successfully.

Steps

1. Enter passenger details.
2. Enter payment information.
3. Submit booking form.

Expected Result

Booking confirmation page should be displayed.

Scenario 4: Validate Booking Confirmation

Description

Verify that booking details are displayed correctly after successful booking.

Expected Result

Confirmation page should display booking information successfully.

Scenario 5: Data-Driven Testing

Description

Verify booking functionality using multiple sets of test data stored in Excel.

Expected Result

All test data sets should execute successfully.

6. Reporting Mechanism

The framework generates multiple reports for execution tracking and analysis.

TestNG Report

Location:

test-output/index.html

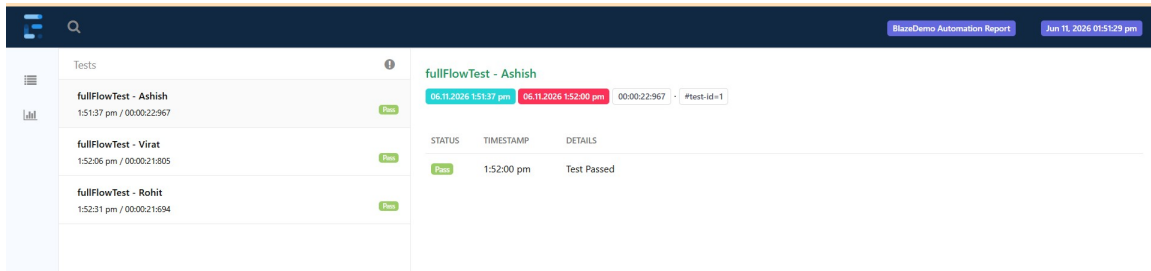
Features:

- Total Tests Executed
- Passed Tests
- Failed Tests
- Skipped Tests
- Execution Summary

Extent Report

Location:

test-output/ExtentReport.html



The screenshot displays the Extent Report dashboard. At the top, there is a header with a search icon and two buttons: "Basic Demo Automation Report" and "Jun 11, 2026 01:51:29 pm". Below the header, the main content area is divided into two sections. On the left, under the heading "Tests", there is a list of three test cases: "fullFlowTest - Ashish" (1:51:37 pm / 0000:22:967), "fullFlowTest - Virat" (1:52:06 pm / 0000:21:805), and "fullFlowTest - Rohit" (1:52:31 pm / 0000:21:694). Each test case has a green "Pass" status indicator. On the right, there is a detailed view for the "fullFlowTest - Ashish" test case. It shows the test name, start time (06.11.2026 1:51:37 pm), end time (06.11.2026 1:52:00 pm), duration (00:00:22:967), and test ID (#test-id-1). Below this, there is a table with three columns: STATUS, TIMESTAMP, and DETAILS. The table contains one row with the status "Pass", timestamp "1:52:00 pm", and details "Test Passed".

Tests	fullFlowTest - Ashish
fullFlowTest - Ashish	1:51:37 pm / 0000:22:967
fullFlowTest - Virat	1:52:06 pm / 0000:21:805
fullFlowTest - Rohit	1:52:31 pm / 0000:21:694

STATUS	TIMESTAMP	DETAILS
Pass	1:52:00 pm	Test Passed

Features:

- Interactive Dashboard
 - Pass/Fail Status
 - Execution Timeline
 - Detailed Logs
 - Test Execution Summary
 - Screenshot Integration
-

7. Screenshot Capture Functionality

The framework automatically captures screenshots during execution.

Screenshots are captured for:

- Flight Search Page
- Flight Selection Page
- Purchase Page
- Confirmation Page
- Failed Test Cases

Benefits:

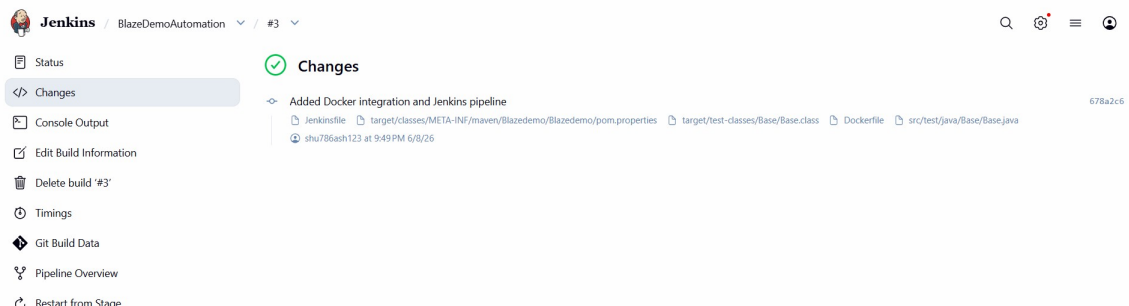
- Easier defect analysis
- Improved reporting
- Better execution evidence

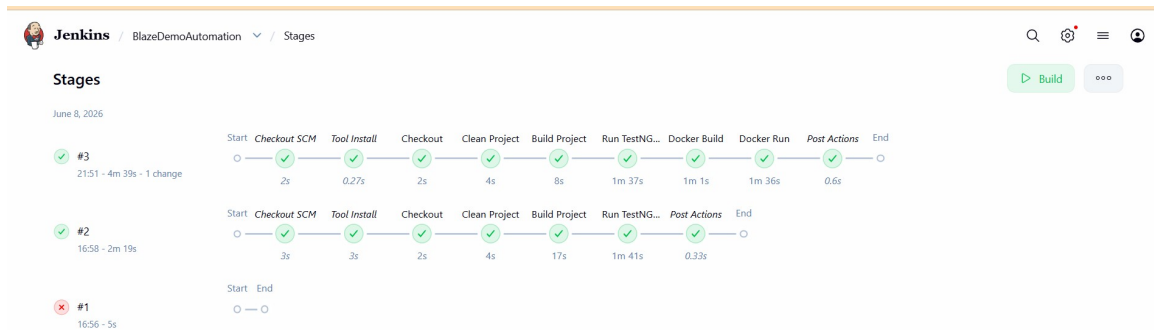
8. Jenkins Integration

The automation framework is integrated with Jenkins Pipeline to enable continuous integration.

Jenkins Pipeline Stages

1. Checkout Source Code
2. Clean Project
3. Build Project
4. Execute TestNG Tests
5. Docker Image Build
6. Docker Container Execution
7. Publish Reports





Benefits

- Automated execution
- Scheduled builds
- Faster feedback
- CI/CD implementation
- Reduced manual effort

9. Docker Integration

Docker has been integrated to provide a consistent execution environment.

Docker Workflow

1. Create Dockerfile
2. Build Docker Image
3. Execute Docker Container
4. Run Automation Tests

Docker Commands

`docker build -t blazedemo-automation .`

`docker run --rm blazedemo-automation`

```
===== TEST COMPLETED SUCCESSFULLY =====
Test Passed: fullFlowTest
Test Execution Finished
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 93.45 s -- in TestSuite
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 01:47 min
[INFO] Finished at: 2026-06-11T15:07:48+05:30
[INFO] -----
D:\wipro-selenium-java-April\BlazeDemoAutomation>
```

```

===== TEST COMPLETED SUCCESSFULLY =====
Test Passed: fullFlowTest
Test Execution Finished
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 103.3
s -- in TestSuite
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
---
[INFO] BUILD SUCCESS
[INFO] -----
---
[INFO] Total time: 01:48 min
[INFO] Finished at: 2026-06-11T09:42:32Z
[INFO] -----
---

D:\wipro-selenium-java-April\BlazeDemoAutomation>docker ps -a
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS          NAMES
D:\wipro-selenium-java-April\BlazeDemoAutomation>

```

Benefits

- Environment consistency
- Easy deployment
- Scalability
- Reduced dependency issues

10. Defects Identified During Testing

Several defects were identified during execution:

Spaces

Recent

BlazeDemo Flight Booking Testing

More spaces

Browse templates

Filters

Dashboards

Teams

Customize sidebar

BD Sprint 1 8 Jun – 18 Jun (6 work items)

000Complete sprint

Execute BlazeDemo Flight Booking Testing and Track Defects

BD-6Passenger Information Displayed on Confirmation Page Does No...

BLAZEDEMO FLIGHT ...IN PROGRESS

Jun 19AS

BD-5Credit Card Number Field Does Not Validate Card Length and Accepts...

BLAZEDEMO FLIGHT ...IN PROGRESS

Jun 19AS

BD-7Booking Completes Successfully Without Entering Mandatory Passen...

BLAZEDEMO FLIGHT ...IN PROGRESS

Jun 19AS

BD-3Booking Completes Successfully Without Entering City and State Deta...

BLAZEDEMO FLIGHT ...IN PROGRESS

Jun 19AS

BD-4Credit Card Month Field Accepts Invalid Values Beyond Valid Month R...

BLAZEDEMO FLIGHT ...IN PROGRESS

Jun 18AS

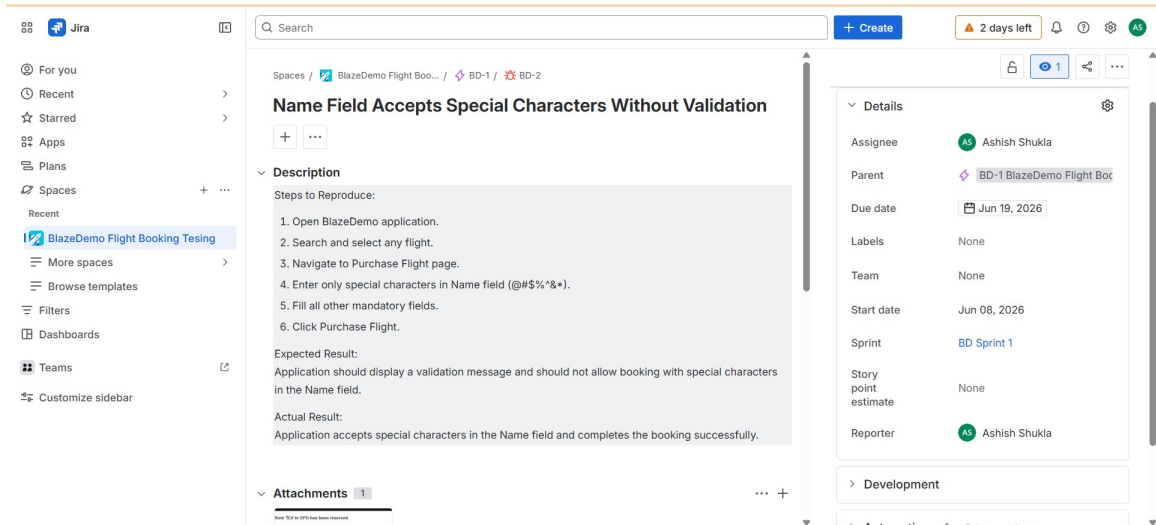
BD-2Name Field Accepts Special Characters Without Validation

BLAZEDEMO FLIGHT ...IN PROGRESS

Jun 19AS

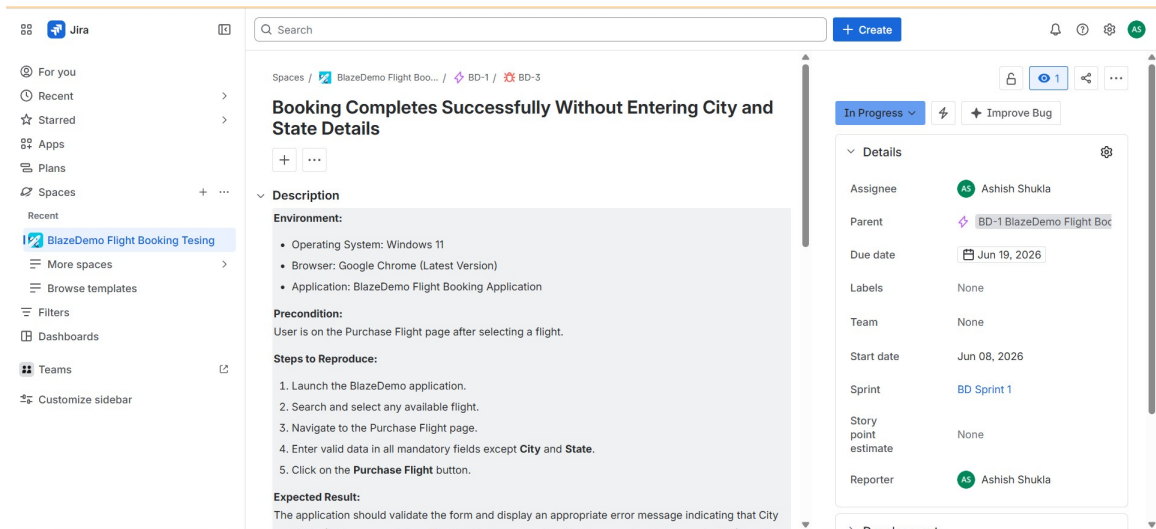
Defect 1

Name field accepts only special characters.



Defect 2

Booking completes without entering City and State.



Defect 3

Expiration month field accepts invalid values.

This screenshot shows a Jira issue in the 'BlazeDemo Flight Booking Testing' space. The issue is titled 'Credit Card Month Field Accepts Invalid Values Beyond Valid Month Range' and is currently in the 'In Progress' state. The left sidebar contains navigation options like 'For you', 'Recent', 'Starred', 'Apps', 'Plans', 'Spaces', 'Filters', 'Dashboards', and 'Teams'. The main content area is divided into sections: 'Description', 'Environment', 'Precondition', and 'Steps to Reproduce'. The 'Environment' section lists 'Operating System: Windows 11', 'Browser: Google Chrome (Latest Version)', and 'Application: BlazeDemo Flight Booking Application'. The 'Precondition' states 'User is on the Purchase Flight page and ready to enter payment details.' The 'Steps to Reproduce' are a 7-step list starting with 'Launch the BlazeDemo application' and ending with 'Click on the Purchase Flight button.' The right sidebar shows issue details: Assignee 'Ashish Shukla', Parent 'BD-1 BlazeDemo Flight Booking', Due date 'Jun 18, 2026', Labels 'None', Team 'None', Start date 'Jun 08, 2026', Sprint 'BD Sprint 1', Story point estimate 'None', and Reporter 'Ashish Shukla'.

Credit Card Month Field Accepts Invalid Values Beyond Valid Month Range

Description

Environment:

- Operating System: Windows 11
- Browser: Google Chrome (Latest Version)
- Application: BlazeDemo Flight Booking Application

Precondition:

User is on the Purchase Flight page and ready to enter payment details.

Steps to Reproduce:

1. Launch the BlazeDemo application.
2. Search and select any available flight.
3. Navigate to the Purchase Flight page.
4. Enter valid passenger information.
5. In the **Expiration Month** field, enter an invalid value such as 13.
6. Fill in the remaining payment details with valid data.
7. Click on the **Purchase Flight** button.

Details

Assignee: Ashish Shukla

Parent: BD-1 BlazeDemo Flight Booking

Due date: Jun 18, 2026

Labels: None

Team: None

Start date: Jun 08, 2026

Sprint: BD Sprint 1

Story point estimate: None

Reporter: Ashish Shukla

Defect 4

Credit card number length is not validated.

This screenshot shows a Jira issue in the 'BlazeDemo Flight Booking Testing' space. The issue is titled 'Credit Card Number Field Does Not Validate Card Length and Accepts Invalid Input' and is currently in the 'In Progress' state. The left sidebar contains navigation options like 'For you', 'Recent', 'Starred', 'Apps', 'Plans', 'Spaces', 'Filters', 'Dashboards', and 'Teams'. The main content area is divided into sections: 'Description', 'Environment', 'Precondition', and 'Steps to Reproduce'. The 'Environment' section lists 'Operating System: Windows 11', 'Browser: Google Chrome (Latest Version)', and 'Application: BlazeDemo Flight Booking Application'. The 'Precondition' states 'User is on the Purchase Flight page and ready to enter payment information.' The 'Steps to Reproduce' are a 7-step list starting with 'Launch the BlazeDemo application' and ending with 'Click on the Purchase Flight button.' The right sidebar shows issue details: Assignee 'Ashish Shukla', Parent 'BD-1 BlazeDemo Flight Booking', Due date 'Jun 19, 2026', Labels 'None', Team 'None', Start date 'Jun 08, 2026', Sprint 'BD Sprint 1', Story point estimate 'None', and Reporter 'Ashish Shukla'.

Credit Card Number Field Does Not Validate Card Length and Accepts Invalid Input

Description

Environment:

- Operating System: Windows 11
- Browser: Google Chrome (Latest Version)
- Application: BlazeDemo Flight Booking Application

Precondition:

User is on the Purchase Flight page and ready to enter payment information.

Steps to Reproduce:

1. Launch the BlazeDemo application.
2. Search and select any available flight.
3. Navigate to the Purchase Flight page.
4. Enter valid passenger details.
5. In the **Credit Card Number** field, enter a card number with fewer or more digits than the standard card length (for example: 12345 or 12345678901234567890).
6. Fill in all remaining required fields with valid data.
7. Click on the **Purchase Flight** button.

Details

Assignee: Ashish Shukla

Parent: BD-1 BlazeDemo Flight Booking

Due date: Jun 19, 2026

Labels: None

Team: None

Start date: Jun 08, 2026

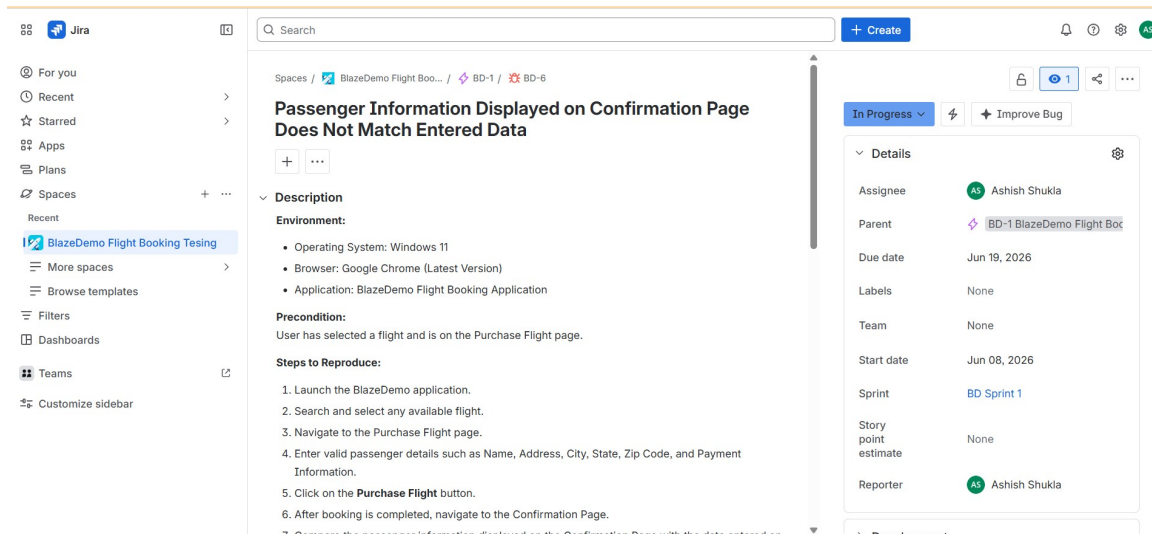
Sprint: BD Sprint 1

Story point estimate: None

Reporter: Ashish Shukla

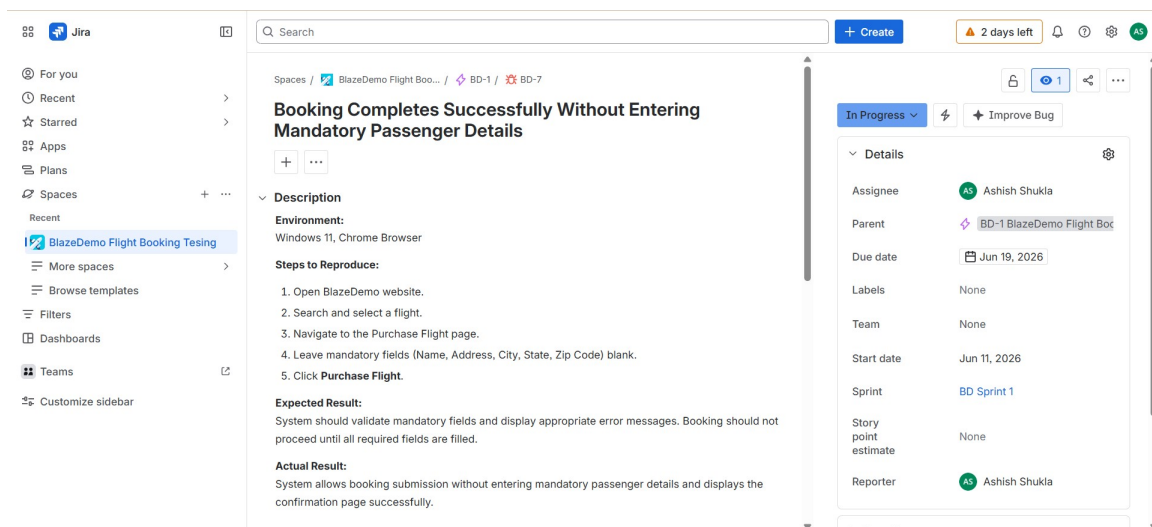
Defect 5

Passenger information mismatch between booking and confirmation pages.



Defect 6

Booking Completes Successfully Without Entering Mandatory Passenger Details



All defects were documented and tracked through Jira.

11. Challenges Faced and Solutions

Challenge 1

Chrome browser execution failure inside Docker container.

Solution

Implemented Chrome Headless Mode using ChromeOptions.

Challenge 2

Jenkins JDK and Maven configuration mismatch.

Solution

Configured correct JDK and Maven tool settings in Jenkins.

Challenge 3

Detailed reporting requirement.

Solution

Integrated Extent Reports using TestNG Listeners.

12. Project Achievements

- Developed complete Selenium Automation Framework.
 - Implemented Page Object Model Architecture.
 - Implemented Data-Driven Testing.
 - Integrated TestNG Reporting.
 - Integrated Extent Reports.
 - Integrated Jenkins CI/CD Pipeline.
 - Integrated Docker Execution.
 - Implemented Screenshot Capture Mechanism.
 - Maintained source code using Git and GitHub.
-

13. Conclusion

The BlazeDemo Flight Booking Automation Framework successfully automates the complete flight booking workflow using Selenium WebDriver and Java. The framework follows industry-standard automation practices such as Page Object Model (POM), Data-Driven Testing, Jenkins CI/CD Integration, Docker Containerization, Screenshot Capture, and Extent Reporting.

The project demonstrates practical implementation of real-world automation testing concepts and provides a scalable, maintainable, and reusable automation solution for web application testing.